

IATSED – Cryptography

Jean-Christophe Deneuille
<jean-christophe.deneuille@enac.fr>

January 8, 2026

During the course, we have seen that a secure cryptosystem can be broken if it is badly implemented (analysis of time and power consumption, fault injection, ...). In this workshop session, we will focus on another aspect of security: the one where the opponent (you) has in his possession an algorithm, and tries to discover its vulnerabilities.

You can use the programming language that suits you, I recommend Python for its agility/flexibility.

Warm-Up

The purpose of this warm-up exercise is to become familiar with symmetrical block cipher schemes using Feistel networks. If you don't understand this sentence, don't worry.

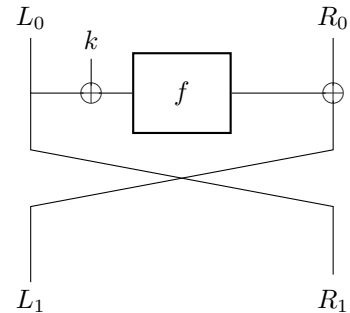
Feistel scheme

We consider two registers of n bits (4 for example) L_0 and R_0 as input, a k key of the same length and a function f of $\{0, 1\}^n$ in $\{0, 1\}^n$.

The output registers L_1 and R_1 are obtained as shown:

$$L_1 \leftarrow R_0 \oplus f(L_0 \oplus k) \quad \text{and} \quad R_1 \leftarrow L_0.$$

A Feistel network is a chain of several Feistel patterns, the functions f and the keys k are called "round" functions and keys and can be different.



Questions

1. How to invert a Feistel pattern (i.e. find L_0 and R_0 from L_1 and R_1 and k)?
2. Does this require the function f to have special properties? If so, which ones? If not, why not?
3. How to invert a 2 rounds Feistel scheme (find L_0 and R_0 from L_2 , R_2 , k_0 and k_1) ?

Exercise 1

Feistel networks are widely used in symmetric cryptosystems, because by involving enough rounds, they allow to obtain good confusion-diffusion properties.

Confusion consists in making information unintelligible (substitution of one letter by another), and diffusion allows to propagate the first property to the whole block.

Description of the target scheme:

Let's consider a simplified cryptosystem $\mathcal{S}$ with a single 4-bit register, admitting an 8-bit key $k = (k_0, k_1)$, and using as function f the one defined by the following table :

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$f(x)$	9	b	c	4	a	1	2	6	d	7	3	8	f	e	0	5

At each round, the input is xored with the round key, then the function f is applied to produce the round output.

1. Implement the round function $f : \text{sb}\text{o}\text{x}(x)$. It takes as input a 4-bit integer x and returns the 4-bit integer $f(x)$ according to the table above.
2. Implement the key derivation function: `split(key)`. From a k key of 8 bits, it returns 2 round subkeys k_0 and k_1 on 4 bits each.

3. Implement the round function : `round(state, roundKey)`. From a state `state` and an input round key `roundKey`, change the state as described above.
4. Implement the encryption function: `encrypt(plaintext, key)`. Assemble the elementary bricks implemented in the previous questions.

Sanity-check : check that `encrypt(0x5, 0xb3) = 0x4`.

Exercise 2

It is not only a question of ensuring confusion and diffusion in order to correctly encrypt, it is also necessary to be able to effectively decrypt...

1. In the case of a Feistel scheme (with 2 inputs/outputs), the function f requires no properties. In our simplified case, which property is necessary to have on f to be able to decrypt?
2. Implement this property.
3. Implement a round of the decryption function.
4. Implement the decryption function: `decrypt(ciphertext, key)`.

Course reminder

There are different attack models depending on the resources available to the opponent:

- attack on ciphertext-only (ciphertext-only, COA),
- known-plaintext attack (KPA),
- chosen-plaintext attack (CPA),
- chosen-ciphertext attack (CCA),
- adaptive chosen-ciphertext attack (CCA2),

Exercise 3

It is generally difficult/impossible to identify a vulnerability of a scheme on a high number of rounds (cf DES, AES, ...). It is simpler to reduce the number of rounds to identify a weakness, then progressively increase the number of rounds to try to exploit the found vulnerability.

1. Let's consider for a moment our S scheme reduced to 1 round. What is the weakest type of attack (the opponent needs a minimum of information) allowing to cryptanalyze this reduced version? Explain the attack.
2. Let's go back to our 2 rounds version. Is the type of attack identified in the previous question still functional? If so, why? If not, why not?

Let x_0 be the input of the encryption algorithm, x_1 the output of the first round (and thus the input of the second round), and x_2 the output of the second round. By finding the right round key in the first turn, we will find the right x_1 , and thus be able to lead the attack on question 1.

3. How many (first) round keys are possible?
4. Why can't we perform a cryptanalysis by this means?

Exercise 4

Although this is not the case in our example, cryptanalysis the full key by brute-force is generally out of reach (keys too large/long).

Let's define an "elementary step" as a round of the encryption algorithm S .

1. How many elementary steps – at most – are needed to brute-force S (i.e. to find a key that, when used in the encryption algorithm, matches our plaintext and ciphertext)?

Let's analyze a little more the encryption algorithm $\mathcal{S}$:

2. How many plaintext messages are possible?
3. How many (complete) keys are possible?
4. How many encrypted messages are possible?
5. What do you conclude?
6. Correct your answer to question 1 of this exercise.

Exercise 5

The previous answer allowed us to demonstrate that the same (plaintext, ciphertext) pair could be obtained with several keys. (If you are not convinced, try to encrypt 1 with the keys 0x47, 0x74, and 0xe3).

1. Write a function `bruteforce(m, c)` which given plaintext message `m` and ciphertext `c` returns the set of possible keys $k = (k_0, k_1)$ such that `encrypt(m, k) = c`.

In order to determine exactly which key was used, we will need more than a single (plaintext, ciphertext) pair.

2. So write a function `known_pairs_gen(k, n)`, which given a key $k = (k_0, k_1)$ and a number n returns a list of n pairs of random plaintext messages with their corresponding ciphertexts.

The cryptanalysis technique we will now study is called linear cryptanalysis [?].

Exercise 6

In the following, we will suppose that it is unreasonable to perform a brute force attack on our scheme $\mathcal{S}$ over two rounds.¹. Since we know (cf. exo 3.1) that it will be easy to find k_1 once we have k_0 , it is on this first round key that we will concentrate our efforts.

Paradigm

A so-called "linear" function has the characteristic of preserving certain mathematical properties (distance, correlation, etc...).

The function f is supposed to ensure the *confusion* of our scheme, i.e. "scrambling" the information, which cannot be linear. (In cryptographic terms we call f a *substitution-box*, abbreviated to S-Box).

The paradigm of linear cryptanalysis is to formulate approximations by linearization, then to validate/reject them by successive tests to learn certain properties on the scheme. The final objective is that the information learned will allow to cryptanalyze the scheme much faster than by brute force.

We are therefore going to focus on singular properties of our Sbox, the simplest of them being the parity of the manipulated states.

1. Create a function `parity(x)` that returns 1 if the number of 1 in the binary writing of x is odd, 0 otherwise.

Typically, the S-boxes of properly designed cryptosystems thwart such tests. However, we can see if there is no correlation for a subset of cases (in other words, only certain positions (or certain bits)).

So let's look for two values $a = \text{mask}_{\text{input}}$ and $b = \text{mask}_{\text{output}}$ such that if we mask (bitwise and) the S-box input with a , the parity of the result is the same than the output of the S-box masked with b .

2. Write a function `nb_parity(a, b)` which, given a and b , calculates the number of x checking :

$$\text{parity}(x \ \& \ a) == \text{parity}(\text{sbox}(x) \ \& \ b).$$

.

Now that we have a "discriminant factor", we want to know which values of a and b maximize this factor.

3. Write a function that returns the value(s) of (a, b) maximizing the function `nb_parity(a, b)` regardless of the value x considered.

Now that we have discriminant values for a and b , let's look for the k_0 key.

¹Outrageously false hypothesis with a purely pedagogical purpose...

Exercise 7

The step of determining a and b is called “S-box linearization”: we have approximated – by a simple function – a complex function. This approximation was supported by a correlation by observation of (plaintext, ciphertext) pairs.

The objective of this exercise is now to exploit the breach opened in the previous exercise.

1. Is our model consistent?

(This last question is a common one for any self-respecting cryptographer/cryptanalyst: we have deduced information from hypotheses. How can we make sure that our assumptions are valid in order to be sure that we can exploit the conclusions)?

2. Use the function `known_pairs_gen(k, n)` to generate 16 (plaintext, ciphertext) pairs.

We now want to assign a score to each of the possible k_0 round keys.

For each possible k_0 , for each known pair (x_0, x_2) of (plaintext, ciphertext), we calculate x_1 the result of the first round of encryption of x_0 with selected k_0 , and we then check if the linear approximation of our Sbox allows to validate the second round, *i.e.* if our binary parity equality holds between x_1 and x_2 .

However be careful, x_2 has been xored (with the real k_1) before going into the Sbox. The xor being an operation linear it won't fundamentally change our property, but it can invert the binary parity and we can thus be finding itself having to count inequalities rather than equalities.

For example, if our linear approximation works in 14 cases out of 16, it may be the case that with the addition of the key, it only works in 2 cases out of 16.

We are therefore looking for a score function that maximizes in both cases (the one where we have a lot of working cases, and the one where we have very few).

3. How would we proceed for such a function?

The candidates to be retained for k_0 are all those who maximize the score of the previous question.

4. Write a candidate selection function for k_0 . This function takes as input the list of pairs (known plaintext, ciphertext), and the previously selected a and b .

Exercise 8

We will now proceed with the actual attack. Let's put the elements end to end.

1. For each k_0 candidate, describe how to find the corresponding k_1 .

We now have a function that predicts the candidate k_1 given the assumption k_0 .

2. Write the function that, given the k_0 candidates, finds the key (if possible).

Exercise 9

Plug in your subroutines to cryptanalyze the S scheme efficiently.

BONUS ;)

Some of your answers are questionable. Which ones do you think?

Let us dwell on question 2 of exercise 3.

1. Reminder: Why is it not possible to cryptanalyze on the first subkey only?
2. What have we implemented to counteract this?

By dedicating a little memory to cryptanalysis, it is possible to significantly/drastically reduce the efforts dedicated to cryptanalysis.

3. Any idea how?

The answer lies in a compromise between memory and CPU time. Let m/c be a (plaintext, ciphertext) couple. Let k'_0 be the key assumed in the first round, and k''_0 the resulting key. In this bonus, we will consider only the $\bar{m}/\bar{c}$ couple, obtained via the $\bar{k}$ key.

Left side :

4. Make an assumption about $\bar{k}_0$
5. Save all `encrypt` ($\bar{m}, \bar{k}_0$) in $\bar{L}$.

Right side :

6. Make an assumption about $\bar{k}_1$
7. Save all `decrypt` ($\bar{c}, \bar{k}_1$) in $\bar{R}$.

Concordance :

8. If there are i and j such that $\bar{L}[i] == \bar{R}[j]$, return “done”, else “fail”.

Analysis of the results :

9. Do you have any matches?
 10. What do you deduce from this?
-